

RAPPORT DE PROJET

Développement d'une Application Java EE

IDENTITÉ

Nom :	Haiti Aya
Classe :	TDIA2
École :	ENSAH
Enseignant :	M.Cherradi
Sujet :	Application Web CRUD Spring MVC avec Hibernate

■ Table des matières

1	Introduction et Architecture	2
2	Étape 1 : Configuration Maven (pom.xml)	2
3	Étape 2 : Configuration (Spring 100% Java)	3
4	Étape 3 : Les Modèles de Données (Entités JPA)	6
5	Étape 4 : Accès aux Données (Repositories)	7
6	Étape 5 : Couche Métier (Services)	8
7	Étape 6 : Sécurité (Intercepteur de Session)	8
8	Étape 7 : Les Contrôleurs (Spring MVC)	9
9	Étape 8 : Présentation (Vues JSP)	11

Introduction et Architecture

Ce projet a pour objectif de développer une application web de gestion de produits en s'appuyant sur l'architecture robuste offerte par **Spring MVC** et **Hibernate (Spring Data JPA)**. L'application utilise une configuration 100% basée sur des classes Java (zéro fichier XML pour la configuration Spring).

- **Configuration Maven** : Gestion des dépendances via le fichier `pom.xml`.
- **Configuration Java** : Remplacement du `web.xml` par des classes de configuration (`AppInitializer`, `WebConfig`, `DatabaseConfig`).
- **Spring Data JPA** : Simplifie drastiquement l'accès aux données grâce aux interfaces `JpaRepository`.
- **Contrôleurs Spring** : Séparent clairement les responsabilités via les annotations `@Controller`.
- **Vues JSP** : Les vues sont stockées dans le répertoire `WEB-INF/views/` et redirigées automatiquement.

Étape 1 : Configuration Maven (pom.xml)

Le fichier `pom.xml` centralise toutes les dépendances requises pour le framework Spring MVC, Hibernate, Spring Data JPA, et les bibliothèques de présentation (JSTL).

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5                             http://maven.apache.org/xsd/maven-4.0.0.xsd"
6     <modelVersion>4.0.0</modelVersion>
7
8     <groupId>com.tp</groupId>
9     <artifactId>springmvc</artifactId>
10    <version>1.0-SNAPSHOT</version>
11    <packaging>war</packaging>
12
13    <name>TP5 - Spring MVC with Hibernate</name>
14
15    <properties>
16        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
17    </properties>
18
19    <dependencies>
20        <!-- Spring Framework Dependencies -->
21        <dependency>
22            <groupId>org.springframework</groupId>
23            <artifactId>spring-core</artifactId>
```

```

28         <version>${spring.version}</version>
29     </dependency>
30     <dependency>
31         <groupId>org.springframework</groupId>
32         <artifactId>spring-webmvc</artifactId>
33         <version>${spring.version}</version>
34     </dependency>
35     <dependency>
36         <groupId>org.springframework</groupId>
37         <artifactId>spring-orm</artifactId>
38         <version>${spring.version}</version>
39     </dependency>
40     <dependency>
41         <groupId>org.springframework.data</groupId>
42         <artifactId>spring-data-jpa</artifactId>
43         <version>2.7.5</version>
44     </dependency>
45
46     <!-- Hibernate & MySQL -->
47     <dependency>
48         <groupId>org.hibernate</groupId>
49         <artifactId>hibernate-core</artifactId>
50         <version>${hibernate.version}</version>
51     </dependency>
52     <dependency>
53         <groupId>mysql</groupId>
54         <artifactId>mysql-connector-java</artifactId>
55         <version>8.0.33</version>
56     </dependency>
57
58     <!-- JSP / JSTL -->
59     <dependency>
60         <groupId>javax.servlet</groupId>
61         <artifactId>javax.servlet-api</artifactId>
62         <version>4.0.1</version>
63         <scope>provided</scope>
64     </dependency>
65     <dependency>
66         <groupId>jstl</groupId>
67         <artifactId>jstl</artifactId>
68         <version>1.2</version>
69     </dependency>
70 </dependencies>
71 </project>

```

Étape 2 : Configuration (Spring 100% Java)

L'application s'initialise via la classe `AppInitializer` qui étend `AbstractAnnotationConfigDispatcher`. Elle remplace le fichier 'web.xml' et charge la configuration applicative.

src/main/java/com/tp/springmvc/config/AppInitializer.java

```
1 package com.tp.springmvc.config;
2
3 import org.springframework.web.servlet.support.
4     ↳ AbstractAnnotationConfigDispatcherServletInitializer;
5
6 public class AppInitializer extends
7     ↳ AbstractAnnotationConfigDispatcherServletInitializer {
8
9     @Override
10    protected Class<?>[] getRootConfigClasses() {
11        return new Class[] { DatabaseConfig.class };
12    }
13
14    @Override
15    protected Class<?>[] getServletConfigClasses() {
16        return new Class[] { WebConfig.class };
17    }
18
19    @Override
20    protected String[] getServletMappings() {
21        return new String[] { "/" };
22    }
23 }
```

Le DatabaseConfig configure Hibernate, le DataSource et le gestionnaire de transactions via les annotations @Configuration et @EnableJpaRepositories.

src/main/java/com/tp/springmvc/config/DatabaseConfig.java

```
1 package com.tp.springmvc.config;
2
3 import org.springframework.context.annotation.*;
4 import org.springframework.data.jpa.repository.config.
5     ↳ EnableJpaRepositories;
6 import org.springframework.transaction.annotation.
7     ↳ EnableTransactionManagement;
8 import org.springframework.jdbc.datasource.DriverManagerDataSource;
9 import org.springframework.orm.jpa.*;
10 import org.springframework.orm.jpa.vendor.HibernateJpaVendorAdapter;
11
12 import javax.persistence.EntityManagerFactory;
13 import javax.sql.DataSource;
14 import java.util.HashMap;
15 import java.util.Map;
16
17 @Configuration
18 @EnableTransactionManagement
19 @EnableJpaRepositories(basePackages = "com.tp.springmvc.repository")
20 public class DatabaseConfig {
21
22     @Bean
23     public DataSource dataSource() {
24         DriverManagerDataSource dataSource = new DriverManagerDataSource
25         ↳ ();
26     }
27 }
```

```

23     dataSource.setDriverClassName("com.mysql.cj.jdbc.Driver");
24     dataSource.setUrl("jdbc:mysql://localhost:3306/springmvc");
25     dataSource.setUsername("root");
26     dataSource.setPassword("Jalal.1234");
27     return dataSource;
28 }
29
30 @Bean
31 public LocalContainerEntityManagerFactoryBean entityManagerFactory(
32     ↪ DataSource dataSource) {
33     LocalContainerEntityManagerFactoryBean em = new
34     ↪ LocalContainerEntityManagerFactoryBean();
35     em.setDataSource(dataSource);
36     em.setPackagesToScan("com.tp.springmvc.entity");
37     em.setJpaVendorAdapter(new HibernateJpaVendorAdapter());
38
39     Map<String, Object> properties = new HashMap<>();
40     properties.put("hibernate.dialect", "org.hibernate.dialect.
41     ↪ MySQL8Dialect");
42     properties.put("hibernate.hbm2ddl.auto", "update");
43     properties.put("hibernate.show_sql", "true");
44     em.setJpaPropertyMap(properties);
45
46     return em;
47 }
48
49 @Bean
50 public JpaTransactionManager transactionManager(EntityManagerFactory
51     ↪ entityManagerFactory) {
52     JpaTransactionManager transactionManager = new
53     ↪ JpaTransactionManager();
54     transactionManager.setEntityManagerFactory(entityManagerFactory);
55     return transactionManager;
56 }
57 }

```

Le WebConfig configure le moteur de résolution de vues JSP et enregistre l'intercepteur de session.

```

src/main/java/com/tp/springmvc/config/WebConfig.java
1 package com.tp.springmvc.config;
2
3 import org.springframework.context.annotation.*;
4 import org.springframework.web.servlet.config.annotation.*;
5 import org.springframework.web.servlet.view.InternalResourceViewResolver;
6
7 @Configuration
8 @EnableWebMvc
9 @ComponentScan(basePackages = "com.tp.springmvc")
10 public class WebConfig implements WebMvcConfigurer {
11
12     @Bean
13     public InternalResourceViewResolver viewResolver() {
14         InternalResourceViewResolver resolver = new

```

```

15     ↪ InternalResourceViewResolver();
16         resolver.setPrefix("/WEB-INF/views/");
17         resolver.setSuffix(".jsp");
18         return resolver;
19     }
20
21     @Override
22     public void addResourceHandlers(ResourceHandlerRegistry registry) {
23         ↪ registry.addResourceHandler("/resources/**").addResourceLocations
24         ↪ ("/resources/");
25     }
26
27     @Override
28     public void addInterceptors(InterceptorRegistry registry) {
29         ↪ registry.addInterceptor(new SessionInterceptor())
30         ↪ .addPathPatterns("/produits/**")
31         ↪ .excludePathPatterns("/login", "/", "/error");
32     }
33 }

```

Étape 3 : Les Modèles de Données (Entités JPA)

Les classes entités sont annotées avec `@Entity` et sont automatiquement mappées en base de données par Spring/Hibernate.

```

src/main/java/com/tp/springmvc/entity/User.java
1 package com.tp.springmvc.entity;
2
3 import javax.persistence.*;
4
5 @Entity
6 @Table(name = "users")
7 public class User {
8     @Id
9     @Column(length = 50)
10    private String login;
11
12    @Column(nullable = false)
13    private String password;
14
15    @Column(nullable = false)
16    private String role;
17
18    // Constructeurs, getters et setters...
19 }

```

```

src/main/java/com/tp/springmvc/entity/Produit.java
1 package com.tp.springmvc.entity;
2
3 import javax.persistence.*;

```

```

4
5 @Entity
6 @Table(name = "produits")
7 public class Produit {
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long idProduit;
11
12    @Column(nullable = false)
13    private String nom;
14
15    @Column(length = 1000)
16    private String description;
17
18    @Column(nullable = false)
19    private Double prix;
20
21    // Constructeurs, getters et setters...
22 }

```

Étape 4 : Accès aux Données (Repositories)

Spring Data JPA remplace l'implémentation fastidieuse des DAO par des interfaces héritant de `JpaRepository`. Le code SQL (via Hibernate) est généré automatiquement.

```

src/main/java/com/tp/springmvc/repository/ProduitRepository.java
1 package com.tp.springmvc.repository;
2
3 import com.tp.springmvc.entity.Produit;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 @Repository
8 public interface ProduitRepository extends JpaRepository<Produit, Long> {
9 }

```

```

src/main/java/com/tp/springmvc/repository/UserRepository.java
1 package com.tp.springmvc.repository;
2
3 import com.tp.springmvc.entity.User;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.Optional;
8
9 @Repository
10 public interface UserRepository extends JpaRepository<User, String> {
11     Optional<User> findByLoginAndPassword(String login, String password);
12 }

```


Étape 5 : Couche Métier (Services)

La logique d'application est encapsulée dans les services. Spring gère l'injection des dépendances via `@Autowired` et la gestion des transactions via `@Transactional`.

src/main/java/com/tp/springmvc/service/ProduitServiceImpl.java

```
1 package com.tp.springmvc.service;
2
3 import com.tp.springmvc.entity.Produit;
4 import com.tp.springmvc.repository.ProduitRepository;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Service;
7 import org.springframework.transaction.annotation.Transactional;
8
9 import java.util.List;
10 import java.util.Optional;
11
12 @Service
13 @Transactional
14 public class ProduitServiceImpl implements ProduitService {
15
16     @Autowired
17     private ProduitRepository produitRepository;
18
19     @Override
20     public Produit addProduit(Produit p) { return produitRepository.save(
21         ↪ p); }
22
23     @Override
24     public List<Produit> getAllProduits() { return produitRepository.
25         ↪ findAll(); }
26
27     @Override
28     public Optional<Produit> getProduitById(Long id) { return
29         ↪ produitRepository.findById(id); }
30
31     @Override
32     public void deleteProduit(Long id) { produitRepository.deleteById(id)
33         ↪ ; }
34
35     @Override
36     public Produit updateProduit(Produit p) { return produitRepository.
37         ↪ save(p); }
38 }
```

Étape 6 : Sécurité (Intercepteur de Session)

Au lieu du filtre classique (`Filter`), on utilise un `HandlerInterceptor` propre à Spring MVC pour sécuriser les routes `/produits/**`.

```

src/main/java/com/tp/springmvc/config/SessionInterceptor.java
1 package com.tp.springmvc.config;
2
3 import org.springframework.web.servlet.HandlerInterceptor;
4 import org.springframework.stereotype.Component;
5 import javax.servlet.http.*;
6
7 @Component
8 public class SessionInterceptor implements HandlerInterceptor {
9
10     @Override
11     public boolean preHandle(HttpServletRequest request,
12         ↪ HttpServletResponse response, Object handler) throws Exception {
13         HttpSession session = request.getSession(false);
14         if (session == null || session.getAttribute("user") == null) {
15             response.sendRedirect(request.getContextPath() + "/login");
16             return false;
17         }
18         return true;
19     }
20 }

```

Étape 7 : Les Contrôleurs (Spring MVC)

L'utilisation de `@Controller` et `@RequestMapping` simplifie considérablement le routage. L'authentification est séparée de la gestion des produits.

```

src/main/java/com/tp/springmvc/controller/AuthController.java
1 package com.tp.springmvc.controller;
2
3 import com.tp.springmvc.entity.User;
4 import com.tp.springmvc.service.UserService;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Controller;
7 import org.springframework.ui.Model;
8 import org.springframework.web.bind.annotation.*;
9 import javax.servlet.http.HttpSession;
10 import java.util.Optional;
11
12 @Controller
13 @RequestMapping("/")
14 public class AuthController {
15
16     @Autowired
17     private UserService userService;
18
19     @GetMapping("login")
20     public String loginPage() { return "login"; }
21
22     @PostMapping("login")
23     public String login(@RequestParam String login, @RequestParam String
24         ↪ password, HttpSession session, Model model) {

```

```

24     Optional<User> user = userService.findByLoginAndPassword(login,
    ↪ password);
25     if (user.isPresent()) {
26         session.setAttribute("user", user.get());
27         return "redirect:/produits";
28     } else {
29         model.addAttribute("erreur", "Login ou mot de passe invalide
    ↪ ");
30         return "login";
31     }
32 }
33
34 @GetMapping("logout")
35 public String logout(HttpSession session) {
36     session.invalidate();
37     return "redirect:/login";
38 }
39 }

```

```

src/main/java/com/tp/springmvc/controller/ProduitController.java
1 package com.tp.springmvc.controller;
2
3 import com.tp.springmvc.entity.*;
4 import com.tp.springmvc.service.ProduitService;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.stereotype.Controller;
7 import org.springframework.ui.Model;
8 import org.springframework.web.bind.annotation.*;
9 import javax.servlet.http.HttpSession;
10
11 @Controller
12 @RequestMapping("/produits")
13 public class ProduitController {
14
15     @Autowired
16     private ProduitService produitService;
17
18     private User getUser(HttpSession session) { return (User) session.
    ↪ getAttribute("user"); }
19
20     @GetMapping("")
21     public String listProduits(HttpSession session, Model model) {
22         User user = getUser(session);
23         model.addAttribute("listeProduits", produitService.getAllProduits
    ↪ ());
24         model.addAttribute("produit", new Produit());
25
26         if ("admin".equals(user.getRole())) {
27             return "admin/index";
28         } else {
29             return "user/list";
30         }
31     }
32 }

```

```

33     @PostMapping("ajouter")
34     public String addProduit(@ModelAttribute Produit produit, HttpSession
↪ session) {
35         produitService.addProduit(produit);
36         return "redirect:/produits";
37     }
38
39     @PostMapping("modifier")
40     public String updateProduit(@ModelAttribute Produit produit,
↪ HttpSession session) {
41         produitService.updateProduit(produit);
42         return "redirect:/produits";
43     }
44
45     @GetMapping("supprimer/{id}")
46     public String deleteProduit(@PathVariable Long id, HttpSession
↪ session) {
47         produitService.deleteProduit(id);
48         return "redirect:/produits";
49     }
50 }

```

Étape 8 : Présentation (Vues JSP)

Les vues utilisent JSTL pour dynamiser les données. Le ViewResolver de Spring sait que "login" correspond à /WEB-INF/views/login.jsp.

```

src/main/webapp/WEB-INF/views/login.jsp
1 <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 <!DOCTYPE html>
3 <html lang="fr">
4 <head>
5     <title>Login - TP5 Spring MVC</title>
6 </head>
7 <body>
8 <div class="login-container">
9     <h2>    Login</h2>
10    <form action="${pageContext.request.contextPath}/login" method="post
↪ ">
11        Login: <input type="text" name="login" required />
12        Mot de passe: <input type="password" name="password" required />
13        <button type="submit" class="login-btn">Se connecter</button>
14    </form>
15 </div>
16 </body>
17 </html>

```

```

src/main/webapp/WEB-INF/views/admin/index.jsp
1 <%@ page contentType="text/html; charset=UTF-8" language="java" %>
2 <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

```

```

3 <!DOCTYPE html>
4 <html lang="fr">
5 <head><title>Gestion des Produits</title></head>
6 <body>
7     <h1>          Gestion des Produits</h1>
8     <span>          Bienvenue, <strong>${sessionScope.user.login}</strong></
↳ span>

9
10    <form action="${empty produit.idProduit ? pageContext.request.
↳ contextPath.concat('/produits/ajouter') : pageContext.request.
↳ contextPath.concat('/produits/modifier')}" method="post">
11        <input type="hidden" name="idProduit" value="${produit.idProduit
↳ }" />
12        Nom: <input type="text" name="nom" value="${produit.nom}"
↳ required />
13        Prix: <input type="number" name="prix" value="${produit.prix}"
↳ required />
14        <button type="submit">${empty produit.idProduit ? '    Ajouter' :
↳ '    Modifier'}</button>
15    </form>
16
17    <table>
18        <c:forEach var="p" items="${listeProduits}">
19            <tr>
20                <td>${p.idProduit}</td>
21                <td>${p.nom}</td>
22                <td>${p.prix}</td>
23                <td>
24                    <a href="${pageContext.request.contextPath}/produits/
↳ editer/${p.idProduit}">Modifier</a>
25                    <a href="${pageContext.request.contextPath}/produits/
↳ supprimer/${p.idProduit}">Supprimer</a>
26                </td>
27            </tr>
28        </c:forEach>
29    </table>
30 </body>
31 </html>

```